

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA1304

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:100 Duration : 1 Hour

CLASS TEST

Code of Conduct:

I Pusalapati Chandu(192372119) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

*action. **Signature of the student:***

Pchandu

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design a PDA to accept strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b', where 'n' is a non-negative integer.

$$i) L = \{a^n b^n\}$$

2. Convert the given context-free grammar (CFG) into a Pushdown Automaton (PDA) that recognizes the same language.

$$E \rightarrow E+E \mid E^*E \mid (E) \mid I$$

$$I \rightarrow Ia \mid Ib \mid IO \mid I1 \mid a \mid b$$

3. Design a TM to accepts the strings with 'n' occurrences of 'a' followed by 'n' occurrences of 'b', where 'n' is a non-negative integer

$$i) L = \{0^n 1^n \mid n \geq 1\}$$

4. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

5. Construct a Turing Machine to recognize the language. Write the logic used for the design and the formal definition

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

SIMATS ENGINEERING

ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. PDA FOR $L = \{ a^n b^n \mid n \text{ IS A NON-NEGATIVE INTEGER} \}$

The question asks for a PDA accepting strings with n occurrences of a followed by n occurrences of b , where n is non-negative.

Aim

To construct a Pushdown Automaton that accepts exactly the strings $a^n b^n$ for $n \geq 0$.

Language

$L = \{ a^n b^n \mid n \geq 0 \} = \{ \epsilon, ab, aabb, aaabbb, \dots \}$

Basic Logic

- Read a 's first and push one stack symbol X for every a .
- When the first b is read, change to the b -reading state.
- For every b , pop one X .
- If an a appears after b processing has started, reject.
- Accept when all input is consumed and the stack returns to the bottom marker.
- Because n can be 0 , the empty string ϵ is also accepted.

PDA Formal Definition

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Component	Value
Q	$\{q_0, q_1, q_f\}$
Σ	$\{a, b\}$
Γ	$\{Z_0, X\}$
q_0	Initial state
Z_0	Initial stack symbol
F	$\{q_f\}$

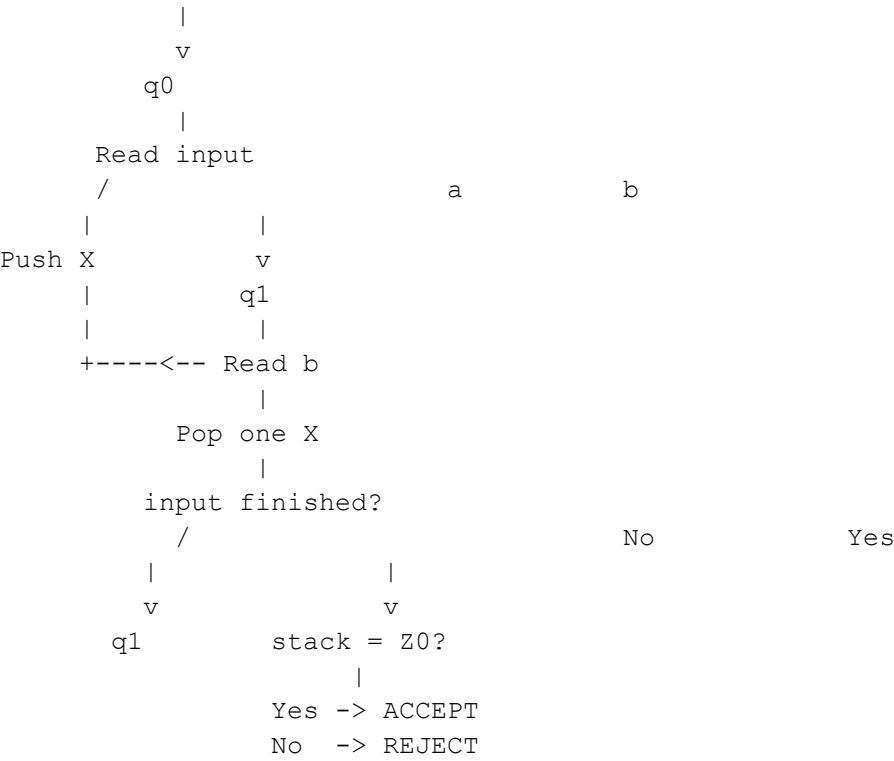
Transition Table

State	Input	Stack Top	Action	Next State
q_0	a	Z_0	Push X above Z_0	q_0
q_0	a	X	Push another X	q_0
q_0	b	X	Pop X	q_1
q_1	b	X	Pop X	q_1
q_1	a	X	No transition $\rightarrow$ reject	—
q_1	end	Z_0	Accept	q_f

q0	end	Z0	Accept ϵ	qf
----	-----	----	-------------------	----

Flowchart

START



Example 1: aabb

Input: aabb

Read a -> push X
Read a -> push X

Stack: XXZ0

Read b -> pop X
Read b -> pop X

Stack: Z0
Input finished
Result: ACCEPT

Example 2: ϵ

Since n is allowed to be 0, $\epsilon = a^0b^0$ belongs to the language. Therefore the PDA must accept the empty input.

Example 3: aab

Two a's require two b's. Only one b is present, so one X remains on the stack. The string is rejected.

Correctness

Every a contributes one X. Every b removes one X. Therefore the PDA can accept only when the number of a's equals the number of b's. The state change after the first b also ensures that all a's occur before all b's.

Conclusion

The PDA correctly recognizes $a^n b^n$ for $n \geq 0$, including ϵ .

2. CONVERT THE GIVEN CFG INTO AN EQUIVALENT PDA

The uploaded question gives the grammar: $E \rightarrow E+E \mid E * E \mid (E) \mid I$ and $I \rightarrow Ia \mid Ib \mid I0 \mid I1 \mid a \mid b$, and asks for a PDA recognizing the same language. □filecite□turn11file0□L24-L27□

Aim

To construct an NPDA that simulates the derivations of the given CFG by storing grammar symbols on the stack and matching terminals with the input.

Given CFG

$E \rightarrow E + E \mid E * E \mid (E) \mid I$
 $I \rightarrow Ia \mid Ib \mid I0 \mid I1 \mid a \mid b$

Construction Principle

1. Push the start variable E onto the stack.
2. If the top of the stack is a variable, replace it using one of its grammar productions.
3. Because there are multiple productions, the PDA uses nondeterministic choices.
4. If the top of the stack is a terminal equal to the next input symbol, consume the input and pop that terminal.
5. When the entire input is consumed and the stack contains only $Z0$, accept.

PDA Components

Component	Definition
States	q_0 = processing, q_f = accepting
Input alphabet	$\{a, b, 0, 1, +, *, (,)\}$
Stack alphabet	$\{Z_0, E, I, +, *, (,), a, b, 0, 1\}$
Initial stack	Z_0

Initial operation	Push E above Z0
Acceptance	Input consumed and stack reduced to Z0

CFG-to-PDA Transition Pattern

For every production $A \rightarrow \alpha$, add an ϵ -transition that replaces A on the stack by α . For every terminal x, add a transition that reads x from the input when x is on top of the stack.

Initial:

```
(q0, w, Z0)
  |
  |  $\epsilon$ , Z0  $\rightarrow$  E Z0
  v
(q0, w, E Z0)
```

If E is on top:

```
E  $\rightarrow$  E+E
E  $\rightarrow$  E*E
E  $\rightarrow$  (E)
E  $\rightarrow$  I
```

If I is on top:

```
I  $\rightarrow$  Ia
I  $\rightarrow$  Ib
I  $\rightarrow$  I0
I  $\rightarrow$  I1
I  $\rightarrow$  a
I  $\rightarrow$  b
```

Terminal matching:

```
input x + stack top x
  |
  v
consume x and pop x
```

When input is empty and stack = Z0:

```
q0  $\rightarrow$  qf
ACCEPT
```

Example: Input a+b

One possible derivation is $E \Rightarrow E+E \Rightarrow I+I \Rightarrow a+b$. The PDA can choose the corresponding grammar productions and then match a, + and b against the input.

Important Note

The grammar is ambiguous because expressions such as $a+b*c$ can have more than one possible parse under the productions $E \rightarrow E+E$ and $E \rightarrow E*E$. The NPDA can still recognize the language because it can nondeterministically choose a valid derivation.

Conclusion

The standard CFG-to-NPDA construction gives an equivalent PDA by placing grammar variables and terminals on the stack, expanding variables using ϵ -moves and consuming terminals by input-matching transitions.

3. TURING MACHINE FOR EQUAL NUMBERS OF 0s AND 1s

The question asks for a TM accepting strings with n occurrences of one symbol followed by n occurrences of the other, but the extracted source shows an OCR conflict: the line introduces a's and b's while the displayed language appears as 0^n1^n . □filecite□turn11file0□L28-L33□

Source-Based Interpretation

The displayed formal expression is most consistently interpreted as $L = \{0^n1^n \mid n \geq 1\}$. The design below therefore uses 0's followed by 1's. If the intended symbols were a and b, replace 0 with a and 1 with b throughout.

Aim

To construct a Turing Machine that accepts exactly 0^n1^n for $n \geq 1$.

Tape Symbols

Symbol	Meaning
0	Unmarked 0
1	Unmarked 1
X	Marked 0
Y	Marked 1
B	Blank

Logic

- Find the leftmost unmarked 0 and change it to X.
- Move right until an unmarked 1 is found and change it to Y.
- Return to the left end.
- Repeat while unmarked 0's remain.
- When no unmarked 0 remains, scan the tape to ensure no unmarked 1 remains.
- Accept if all symbols are marked correctly; otherwise reject.

State Flow

```
q0: Find unmarked 0
|
| mark 0 -> X
v
q1: Find unmarked 1
|
| mark 1 -> Y
v
q2: Return to left
|
+----- unmarked 0? ----- YES ----> q0
|
NO
v
q3: Check remaining 1s
|
+----- unmarked 1 ----> REJECT
|
none
v
ACCEPT
```

Example: 0011

Input: 0011

Round 1:
X0Y1

Return left.

Round 2:
XXYY

No unmarked 0 or 1.
Result: ACCEPT

Rejected Example: 001

The first 0 can be paired with a 1, but the second 0 has no remaining 1. Therefore the machine rejects.

Formal Definition

A suitable TM can be represented as $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where $\Sigma = \{0,1\}$, $\Gamma = \{0,1,X,Y,B\}$, q_0 is the initial state, B is blank, and F contains the accepting state q_{acc} .

Correctness

Each round pairs exactly one 0 with one 1. Thus acceptance occurs only when the counts are equal. Because the machine also expects all 0's before all 1's, the recognized language is exactly $0^n 1^n$ for $n \geq 1$.

Conclusion

The marking technique provides a simple TM construction for equal numbers of 0's followed by 1's.

4. PDA FOR a's FOLLOWED BY b's, WITH TWICE AS MANY b's

The question asks for a PDA where a's are followed by b's and the number of b's must be twice the number of a's. The extracted mathematical notation is partially garbled, but the verbal requirement is clear.

Aim

To construct a PDA accepting $L = \{a^n b^{2n} \mid n \geq 1\}$.

Basic Idea

For every a, push TWO stack symbols X. Then, for every b, pop ONE X. If there are exactly twice as many b's as a's, all X symbols will be removed exactly at the end of the input.

Algorithm

12. Read an a and push two X symbols.
13. Repeat for every a.
14. When the first b is read, move to the b-processing state.
15. For every b, pop one X.
16. Reject if a b is encountered when no X remains.
17. Reject if an a appears after b processing begins.
18. Accept when input ends and the stack returns to Z_0 .

PDA Components

Component	Value
States	q_0, q_1, q_f
Input alphabet	$\{a,b\}$
Stack alphabet	$\{Z_0, X\}$
Initial state	q_0
Final state	q_f

Transition Logic

State	Input	Stack	Operation	Next
q0	a	Z0	Push XX	q0
q0	a	X	Push XX	q0
q0	b	X	Pop X	q1
q1	b	X	Pop X	q1
q1	a	X	Reject	—
q1	b	Z0	Reject	—
q1	end	Z0	Accept	qf

Example: aabbbb

Input: aabbbb

First a -> push XX

Second a -> push XX

Stack contains 4 X symbols.

b -> pop X

b -> pop X

b -> pop X

b -> pop X

Stack returns to Z0.

Result: ACCEPT

Rejected Example: aabb

One a requires two b's. Two a's require four b's. Since only two b's are present, two X symbols remain and the machine rejects.

Correctness

Each a contributes exactly two X symbols, so n a's create $2n$ X symbols. Each b removes exactly one X. Therefore, exactly $2n$ b's are required for the stack to become empty at the correct time.

Conclusion

The PDA recognizes $a^n b^{2n}$ for $n \geq 1$.

5. TURING MACHINE FOR THE LANGUAGE IN QUESTION 5

The extracted text shows the language expression for Question 5 as garbled: “{ ... | $n \geq 1$ }”. The source does not preserve the actual symbols or pattern, so an exact language-specific TM cannot be safely constructed from the file alone. □filecite□turn11file0□L37-L39□

Important Source Limitation

Because the language itself is unreadable in the supplied document extraction, the correct response is to avoid inventing a language. The general TM construction method and formal definition are given below. If the original page image clearly shows the missing language, that expression should be inserted into this section and the transition table finalized accordingly.

General TM Design Logic

19. Identify the exact input pattern and alphabet.
20. Decide what must be counted, matched, copied or checked.
21. Choose marker symbols for processed input.
22. Use states to represent different phases of the algorithm.
23. Use the tape to store information that cannot fit in finite control.
24. Repeat scanning and marking until the required condition is verified.
25. Enter the accepting state only after the complete condition has been checked.
26. Reject if an invalid symbol, wrong order or unmatched symbol is found.

General TM Structure

```
START
|
v
q0: Validate input pattern
|
v
q1: Find next unprocessed symbol
|
v
q2: Mark / compare / process it
|
v
q3: Return to required tape position
|
+---- more symbols? ---- YES ----> q1
|
NO
```

```

v
q4: Final verification
|
+----- condition satisfied ---> ACCEPT
|
+----- condition violated ----> REJECT

```

Formal Definition

A standard Turing Machine is $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where Q is the finite set of states, Σ is the input alphabet, Γ is the tape alphabet, δ is the transition function, q_0 is the initial state, B is the blank symbol and F is the set of accepting states.

Transition Form

$$\delta(q, a) = (p, b, D)$$

Meaning:

q = current state

a = symbol read

p = next state

b = symbol written

D = movement direction (L or R)

Why the Exact TM Cannot Be Given

A Turing Machine is defined around the language it must recognize. If the language expression is missing, the required comparisons and transitions are unknown. Providing a specific machine would therefore be an unsupported assumption.

What Is Needed

- The original Question 5 language expression in readable form.
- Alternatively, a clear screenshot/image of Question 5.
- Once the language is visible, the exact states, tape symbols, transitions, sample traces and correctness proof can be completed.

Conclusion

The general TM design and formal definition are provided, but the exact machine for Q5 requires the unreadable language expression from the source.

